# Questions 3 & 4 — Deadlock Prevention Write-Up

Manav Singla, 2402MC02 — resourceorder_bad.c / resourceorder_fixed.c / syncdeadlock.c, xv6-riscv

## Question 3 — why consistent lock ordering eliminates circular wait

In `resourceorder_bad.c`, Process A takes Lock1 then requests Lock2, while Process B takes Lock2 then requests Lock1. With a deliberate `pause(10)` between each process's two acquisitions, both processes reliably end up holding one lock while blocked on the other: A holds Lock1 waiting on Lock2, B holds Lock2 waiting on Lock1. This is a 2-node cycle in the wait-for graph (A → B → A), so all four of Coffman's necessary conditions for deadlock are met (mutual exclusion, hold-and-wait, no preemption, circular wait) and the run genuinely hangs — confirmed by running it under a 12-second bounded timeout, after which nothing further is ever printed and the harness has to force-quit qemu.

`resourceorder_fixed.c` changes nothing about *what* each process needs, only the *order* it asks for them: every process now takes Lock1 before Lock2, unconditionally. Whichever process loses the race for Lock1 simply blocks on Lock1 — a lock the winner is not itself waiting on — so it can never contribute a wait-for edge back to the winner. In general: if every process in the system is constrained to acquire a set of shared resources in the same total order, then a wait-for edge P → Q can only exist when P is blocked on a resource that is later than something P already holds, and Q holds it. For a cycle to exist, some edge in it would have to point backwards along that order — which is exactly the requirement resource ordering forbids. Circular wait becomes structurally impossible, not just unlikely, which is why the same workload now completes every time (see the two output logs).

## Question 4 — strategy used for the multi-resource dining problem

`syncdeadlock.c` models 5 processes each needing 2 of 3 pooled resource types (printer×2, scanner×1, disk×2), implemented directly as counting semaphores. The strategy used is the same **resource ordering** technique from Q3, generalised from 2 locks to 3 resource types: printer, scanner and disk are given a fixed global index (0, 1, 2), and every process's two required resources are always acquired in increasing index order, regardless of which two types that particular process happens to need.

This was chosen over a Banker's-Algorithm-style admission check because the resource counts here are small, fixed, and each process's maximum need (exactly 2 units, one of two specific types) is already known and constant — ordering gives the same deadlock-freedom guarantee as Q3 with none of the bookkeeping a general admission check would need, and it composes with the counting semaphores directly: each `sem_wait` already blocks correctly if a pool is exhausted, so ordering only has to fix the *sequence* in which processes ask.

The proof is the same structural argument as Q3, extended to 3 resource classes instead of 2: a wait-for cycle would require some process to hold a higher-ordered resource while blocked on a lower-ordered one already held by a process blocked on the first process's resource — impossible once every acquisition path is monotonically increasing in the same fixed order. The real run (`output_logs/syncdeadlock.txt`) shows heavy contention specifically on the single-instance scanner (4 of the 5 processes need it) with processes correctly blocking and being released in turn, yet all 5 complete all 3 cycles and the run ends with `All processes completed, no deadlock` — no process is ever permanently blocked, and the semaphore counts (2, 1, 2) guarantee no pool is ever over-allocated.